



Managing Data

Storage, PVCs & StatefulSets

Intermediate Kubernetes — Module 3 of 13





Module Agenda

Concepts

- Kubernetes storage model
- Volumes, PVs, and PVCs
- Access modes & reclaim policies
- StorageClasses & dynamic provisioning

Patterns & Practice

- StatefulSets deep dive
- AWS EBS & EFS CSI drivers
- Database-on-K8s patterns
- Volume expansion & snapshots

Goal: Understand when and how to persist data in Kubernetes, and choose the right storage abstraction for your workload.



The Kubernetes Storage Model

Ephemeral by default — persistent by design

Containers Are Ephemeral

- Every container starts with a **fresh filesystem** from its image layers
- Writes go to a **writable container layer** (copy-on-write)
- When the container restarts, **all writes are lost**
- Pod rescheduling to a different node = completely new filesystem

This means: Logs written to disk, temp files, cached data, database files — all gone when a pod restarts. This is by design, not a bug.



When You Need Persistence

Databases

- PostgreSQL, MySQL, MongoDB
- Redis (AOF mode)

Message Queues

- Kafka, RabbitMQ
- NATS JetStream

File Storage

- User uploads, ML artifacts
- Shared config and build caches

The Rule of Thumb

If your workload writes data that must survive a pod restart, you need persistent storage. If it only reads data or writes ephemeral data, the default model works fine.



Volumes vs PersistentVolumes vs PVCs

Three Layers of Storage Abstraction

- **Volume** — a directory accessible to containers in a pod; lifecycle tied to the pod
- **PersistentVolume (PV)** — a cluster-level storage resource with its own lifecycle, independent of any pod
- **PersistentVolumeClaim (PVC)** — a developer's request for storage; binds to a PV

Analogy: PV = the actual hard drive provisioned by the platform team. PVC = a ticket requesting "I need 10Gi of fast storage." Kubernetes matches claims to volumes automatically.



Volume Types Overview

Type	Persists?	Use Case
emptyDir	Pod lifetime only	Scratch space, shared data between containers in a pod
hostPath	Node lifetime	Node-level logs, DaemonSet workloads (avoid for app workloads)
configMap	Cluster lifetime	Configuration files mounted as volumes
secret	Cluster lifetime	Certificates, credentials mounted as files
persistentVolumeClaim	Until PV deleted	Databases, stateful apps, durable file storage



emptyDir & hostPath in Practice

```
# emptyDir - shared scratch space between containers
spec:
  containers:
    - name: app
      volumeMounts:
        - { name: shared-logs, mountPath: /var/log/app }
    - name: log-shipper
      volumeMounts:
        - { name: shared-logs, mountPath: /var/log/app, readOnly: true }
  volumes:
    - name: shared-logs
      emptyDir: { sizeLimit: 500Mi }
```




PersistentVolume (PV) Deep Dive

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-ebs-01
spec:
  capacity: { storage: 50Gi }
  volumeMode: Filesystem
  accessModes: ["ReadWriteOnce"]
  persistentVolumeReclaimPolicy: Retain
  storageClassName: gp3
  csi:
    driver: ebs.csi.aws.com
    volumeHandle: vol-0abc123def456789
    fsType: ext4
```

- **capacity** — how much storage the volume provides
- **accessModes** — who can mount and how (RWO, ROX, RWX)
- **reclaimPolicy** — what happens when the PVC is deleted
- **storageClassName** — links to a StorageClass for dynamic provisioning



PersistentVolumeClaim (PVC) Deep Dive

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-data
  namespace: my-team
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: gp3
  resources:
    requests:
      storage: 50Gi
```

```
# Reference in a Pod spec
spec:
  containers:
    - name: postgres
      image: postgres:16
      volumeMounts:
        - name: data
          mountPath: /var/lib/postgresql/data
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: postgres-data
```

PVC Binding Lifecycle

How PVCs Find Their PVs

1. **Pending** — PVC created, waiting for a matching PV
2. **Bound** — PVC matched to a PV (by size, access mode, StorageClass)
3. **In Use** — Pod mounts the PVC; volume is attached to the node
4. **Released** — Pod deleted, PVC deleted; PV enters Released state
5. **Reclaimed** — PV is deleted, retained, or recycled per reclaim policy

```
# Inspect PVC status
kubectl get pvc postgres-data
# NAME          STATUS  VOLUME   CAPACITY  ACCESS MODES  STORAGECLASS
# postgres-data  Bound   pvc-a1b2  50Gi      RW0           gp3

kubectl describe pvc postgres-data | grep -A2 Events
```

Access Modes Explained

Mode	Abbreviation	Description	Typical Backend
ReadWriteOnce	RWO	Single node can mount read-write	EBS, local SSD
ReadOnlyMany	ROX	Many nodes can mount read-only	EFS, NFS
ReadWriteMany	RWX	Many nodes can mount read-write	EFS, NFS, CephFS



Reclaim Policies

Retain

- PV and data preserved after PVC deletion
- Manual cleanup required
- **Recommended for databases**

Delete

- PV and storage permanently removed
- Default for dynamic provisioning
- **Suitable for ephemeral workloads**

Recycle (Deprecated)

- Deprecated; do not use
- Use dynamic provisioning instead



StorageClasses & Dynamic Provisioning

Automate volume creation with declarative storage tiers

StorageClass Anatomy

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp3
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  encrypted: "true"
  fsType: ext4
reclaimPolicy: Retain
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer
```

- **provisioner** — which CSI driver creates the volumes
- **parameters** — passed to the provisioner (EBS type, encryption, etc.)
- **volumeBindingMode** — `WaitForFirstConsumer` delays binding until a pod is scheduled (AZ-aware)
- **allowVolumeExpansion** — enables online resizing of PVCs

AWS EBS CSI Driver on EKS

gp3 — General Purpose

```
kind: StorageClass
metadata: { name: gp3 }
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
  encrypted: "true"
reclaimPolicy: Retain
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
```

- 3,000 IOPS, 125 MB/s baseline
- Best for most workloads

io2 — High Performance

```
kind: StorageClass
metadata: { name: io2-high-perf }
provisioner: ebs.csi.aws.com
parameters:
  type: io2
  iopsPerGB: "50"
  encrypted: "true"
reclaimPolicy: Retain
volumeBindingMode: WaitForFirstConsumer
```

- Up to 64,000 IOPS, 99.999% durability
- Latency-sensitive databases



AWS EFS CSI Driver for Shared Storage

```
kind: StorageClass
metadata: { name: efs-sc }
provisioner: efs.csi.aws.com
parameters:
  provisioningMode: efs-ap
  fileSystemId: fs-0abc123def456789
  directoryPerms: "700"
  # ... gidRangeStart, gidRangeEnd
mountOptions: [tls]
```

EFS Strengths:

ReadWriteMany, auto-scaling capacity, cross-AZ, shared access across pods and nodes

EFS Trade-offs: Higher latency than EBS, higher cost per GB, not suitable for database workloads

Dynamic Provisioning Workflow

What Happens When You Create a PVC

1. Developer creates a **PVC** referencing a StorageClass
2. PVC stays **Pending** (WaitForFirstConsumer mode)
3. Pod is created referencing the PVC; scheduler places the pod on a node
4. StorageClass provisioner **creates the volume** in the correct AZ
5. A **PV is automatically created** and bound to the PVC
6. The volume is **attached** to the node and **mounted** into the pod

```
# Watch the provisioning happen in real time
kubectl get pvc -w
kubectl get pv
kubectl describe pvc my-claim # Check events for provisioning status
```



EBS vs EFS — Choosing the Right Backend

Criteria	EBS (gp3/io2)	EFS
Access Mode	ReadWriteOnce	ReadWriteMany
Performance	Low latency, high IOPS	Higher latency, throughput-oriented
Availability	Single AZ	Multi-AZ
Use Cases	Databases, caches, single-writer	Shared files, CMS, ML datasets
Cost Model	Per GB provisioned	Per GB stored (scales automatically)

Decision rule: If one pod writes, use EBS. If multiple pods need concurrent read-write access, use EFS.



StatefulSets

Managing workloads that need stable identity and persistent storage

StatefulSets vs Deployments

Deployment

- Pods are interchangeable with random names
- Shared PVC (if any), parallel scaling
- No startup/shutdown ordering
- **Use for:** stateless APIs, web servers, workers

StatefulSet

- Stable identities with predictable names (db-0, db-1)
- Per-replica PVC via volumeClaimTemplates
- Ordered scaling, startup, and shutdown
- **Use for:** databases, Kafka, ZooKeeper

Key distinction: Use a Deployment unless your workload requires stable identity, ordered operations, or per-replica persistent storage.

Most workloads are Deployments



StatefulSet YAML Anatomy

```
apiVersion: apps/v1
kind: StatefulSet
metadata: { name: postgres }
spec:
  serviceName: postgres-headless # Required: headless Service
  replicas: 3
  selector: { matchLabels: { app: postgres } }
  template:
    metadata: { labels: { app: postgres } }
    spec:
      # ... container spec, ports, volumeMounts ...
  volumeClaimTemplates: # Per-replica PVCs
  - metadata: { name: data }
    spec:
      accessModes: ["ReadWriteOnce"]
      storageClassName: gp3
      resources: { requests: { storage: 50Gi } }
```



Stable Network Identities

```
# Headless Service – no ClusterIP, provides DNS for each pod
apiVersion: v1
kind: Service
metadata:
  name: postgres-headless
spec:
  clusterIP: None      # This makes it "headless"
  selector:
    app: postgres
  ports:
  - port: 5432
```

DNS Records Created

- postgres-0.postgres-headless.my-ns.svc.cluster.local
- postgres-1.postgres-headless.my-ns.svc.cluster.local
- postgres-2.postgres-headless.my-ns.svc.cluster.local

Why this matters: Applications can connect to a specific replica by name. The primary can always be reached at postgres-0, regardless of which node it runs on.



VolumeClaimTemplates

How VolumeClaimTemplates Work

Each StatefulSet replica gets its own PVC, named using the pattern:

`<volumeClaimTemplate name>-<statefulset name>-
<ordinal>`

```
# For a StatefulSet named "postgres" with 3 replicas:  
kubectl get pvc  
# NAME          STATUS   VOLUME   CAPACITY   STORAGECLASS  
# data-postgres-0 Bound    pvc-aaa   50Gi       gp3  
# data-postgres-1 Bound    pvc-bbb   50Gi       gp3  
# data-postgres-2 Bound    pvc-ccc   50Gi       gp3
```


12
34

Ordered Deployment & Scaling

Scale Up (0 → 3)

1. Create postgres-0, wait until Ready
2. Create postgres-1, wait until Ready
3. Create postgres-2, wait until Ready

Scale Down (3 → 1)

1. Terminate postgres-2, wait until gone
2. Terminate postgres-1, wait until gone
3. postgres-0 remains

Why ordering matters: Databases initialize the primary (ordinal 0) first, then join replicas one at a time. Unordered startup can cause split-brain.

StatefulSet Update Strategies

RollingUpdate (Default)

```
spec:
  updateStrategy:
    type: RollingUpdate
    rollingUpdate:
      partition: 0
```

- Updates pods in reverse ordinal order
- Waits for each to be Ready

Partition-Based Canary

```
spec:
  updateStrategy:
    type: RollingUpdate
    rollingUpdate:
      partition: 2 # Only update pods >= 2
```

- Pods with ordinal $\geq$ partition get new version
- Lower partition gradually to roll out

Canary example: Set `partition=2` on a 3-replica set → only pod-2 updates. Validate, then lower to 1, then 0.



Real-World Storage Patterns

Recommended approaches for stateful workloads on EKS

PostgreSQL on Kubernetes

```
kind: StatefulSet
metadata: { name: postgresql }
spec:
  serviceName: postgresql-headless
  replicas: 3
  # ... selector, labels
  template:
    spec:
      containers:
        - name: postgresql
          image: postgres:16
          env:
            - { name: PGDATA, value: /var/lib/postgresql/data/pgdata }
            # ... password from Secret, ports, resources
          volumeMounts: [{ name: data, mountPath: /var/lib/postgresql/data }]
      volumeClaimTemplates: # gp3, 100Gi RW0 per replica
      # ...
```

Database Operators — The Recommended Path

Why Use an Operator Instead of Raw StatefulSets?

A raw StatefulSet gives you storage and identity. An operator adds the operational logic databases need to run reliably.

CloudNativePG

- PostgreSQL-specific with automated failover
- Continuous backup to S3 and PITR

Percona Operators

- MySQL, MongoDB, PG support
- Multi-cluster replication and backups

Strimzi

- Apache Kafka with topic management
- TLS automation, rack-aware replication



Volume Expansion — Resizing PVCs

```
# Check if the StorageClass allows expansion
kubectl get sc gp3 -o jsonpath='{.allowVolumeExpansion}'
# true

# Edit the PVC to request more storage
kubectl patch pvc data-postgres-0 -p \
  '{"spec": {"resources": {"requests": {"storage": "100Gi"}}}}'

# Monitor the resize operation
kubectl get pvc data-postgres-0 -w
kubectl describe pvc data-postgres-0 | grep Conditions
```

Volume Snapshots

VolumeSnapshotClass

```
# VolumeSnapshotClass – defines the snapshot provisioner
kind: VolumeSnapshotClass
metadata: { name: ebs-snapclass }
driver: ebs.csi.aws.com
deletionPolicy: Retain
```

Create Snapshot

```
# VolumeSnapshot – take a snapshot of a PVC
kind: VolumeSnapshot
metadata: { name: postgres-snap-20260313 }
spec:
  volumeSnapshotClassName: ebs-snapclass
  source:
    persistentVolumeClaimName: data-postgres-0
```

Restore from Snapshot

```
# Restore from snapshot – create a new PVC with dataSource
kind: PersistentVolumeClaim
metadata: { name: postgres-restored }
spec:
  accessModes: ["ReadWriteOnce"]
  storageClassName: gp3
  resources: { requests: { storage: 100Gi } }
  dataSource:
    name: postgres-snap-20260313
    kind: VolumeSnapshot
    apiGroup: snapshot.storage.k8s.io
```



Storage Best Practices for EKS

Configuration

- Use gp3 as the default StorageClass
- Always set `WaitForFirstConsumer` binding mode
- Enable `allowVolumeExpansion`
- Encrypt all volumes (`encrypted: "true"`)
- Set `Retain` reclaim policy for stateful workloads

Operations

- Monitor PVC usage with Prometheus metrics
- Automate volume snapshots for backups
- Use operators for database workloads
- Test failover and recovery procedures
- Tag EBS volumes for cost allocation

Storage Troubleshooting Guide

Symptom	Likely Cause	Fix
PVC stuck in Pending	No matching StorageClass or CSI driver not installed	<code>kubectl describe pvc</code> — check events
Pod stuck in ContainerCreating	Volume attachment failure (AZ mismatch)	Use <code>WaitForFirstConsumer</code>
Volume attach timeout	EBS volume stuck attaching to a different node	Force detach via AWS console or CLI
Filesystem read-only	Disk full or filesystem errors	Expand PVC or run <code>fsck</code>
StatefulSet pods not starting	Previous pod not yet Ready (ordering)	Check <code>pod-0</code> health first



Lab 3: Storage and StatefulSets

Hands-on: Provision, persist, and manage stateful workloads

- Explore the existing gp2 StorageClass and create a PVC
- Deploy a pod with the PVC and verify dynamic provisioning
- Deploy an nginx StatefulSet with volumeClaimTemplates
- Verify stable network identities, DNS, and per-replica storage
- Test pod deletion and volume reattachment
- Scale the StatefulSet and expand a PVC

Duration: ~40-50 minutes



Module 3 — Key Takeaways

- **Kubernetes storage is ephemeral by default** — persistence requires explicit configuration via PVs and PVCs
- **PVC is the developer interface** — declare what you need, let the platform provision it
- **StorageClasses enable self-service** — dynamic provisioning eliminates manual volume management
- **EBS = single-writer, EFS = shared storage** — choose based on access pattern
- **StatefulSets provide stable identity** — ordered operations, persistent names, per-replica storage
- **Use operators for databases** — they encode operational expertise as code
- **Always set Retain reclaim policy** — protect against accidental data loss

? Questions & Discussion

What stateful workloads does your team run today, and how do you currently manage their storage?

Up Next: Module 4 — Services & Service Discovery